KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

Git 기본 흐름, 눈으로 보기

add · commit · push · pull 할 때
내 파일은 어디로 갈까요?

작업 폴더 → 대기실 → 내 저장소 → 원격 저장소 (GitHub)

오늘의 순서

1
개념 잡기
4개의 공간
용어

3–8쪽

2
혼자 기록하기
init · add · commit
커밋 메시지 규칙

9–18쪽

3
GitHub과 연결
remote add
push

19–23쪽

4
함께 작업하기
clone · fetch · pull

24–28쪽

5
되돌리기
reset · revert
reflog

29–35쪽

6
막힐 때
status 읽기
자주 나는 에러

36–42쪽

마지막 43쪽은 한 장 요약이에요.

왜 Git을 쓸까요?

Git 없이

보고서_최종.docx

보고서_최종_수정.docx

보고서_진짜최종.docx

보고서_진짜최종_v3.docx

어느 게 최신이지?

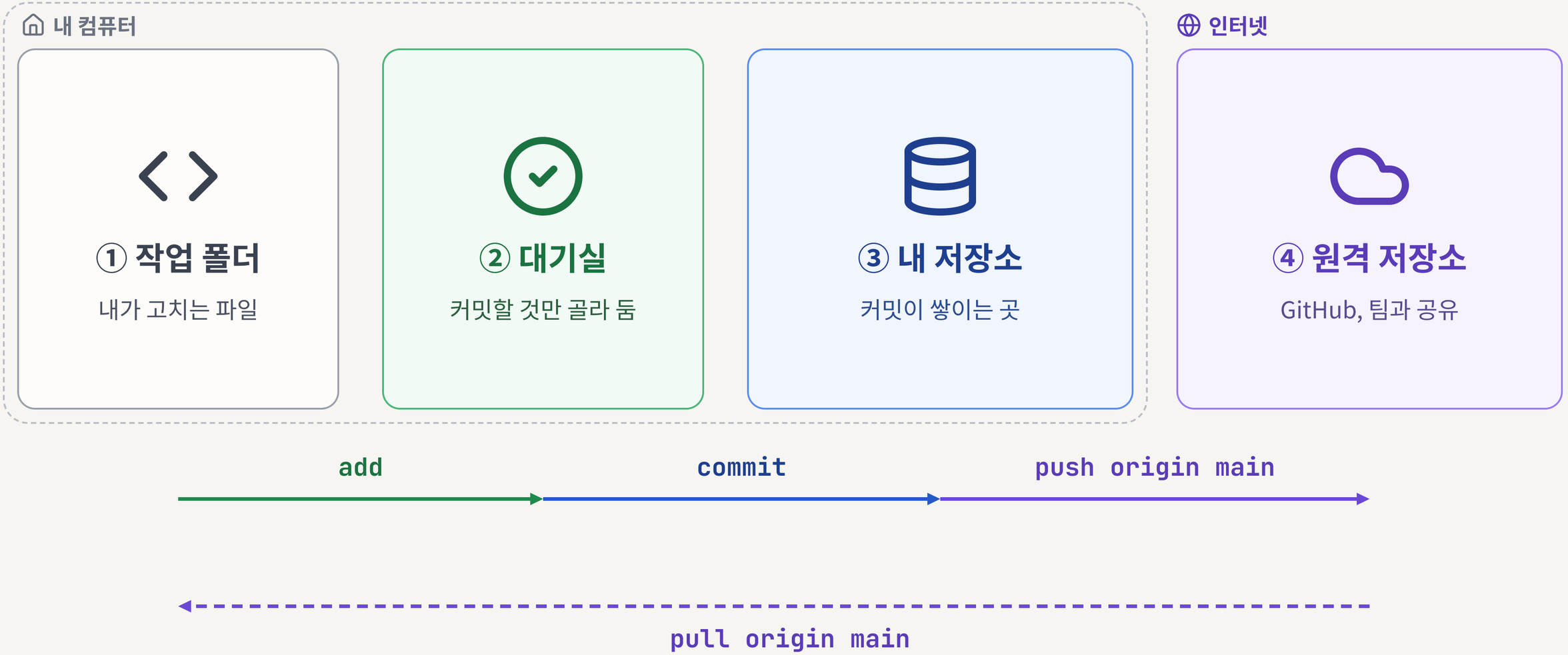
Git과 함께

보고서.docx

- **e73617d** 팀장님 피드백 반영
- **a7c21f4** 오타 수정
- **538dd06** 첫 초안

파일은 하나, 기록은 여러 개

Git에는 4개의 공간이 있어요



택배로 비유하면



add

상자에
담기



commit

봉하고
송장 붙이기



remote add

받는 주소
등록



push

발송



pull

도착한 택배
받아서 풀기

커밋 메시지 = 송장 · 커밋 해시 = 운송장 번호

용어 ① 파일의 상태

상태	뜻	git status에 보이는 말
Untracked	처음 보는 새 파일	Untracked files:
Modified	고쳤지만 add 전	Changes not staged for commit:
Staged	대기실에 올라감	Changes to be committed:
Committed	마지막 커밋과 같음	working tree clean

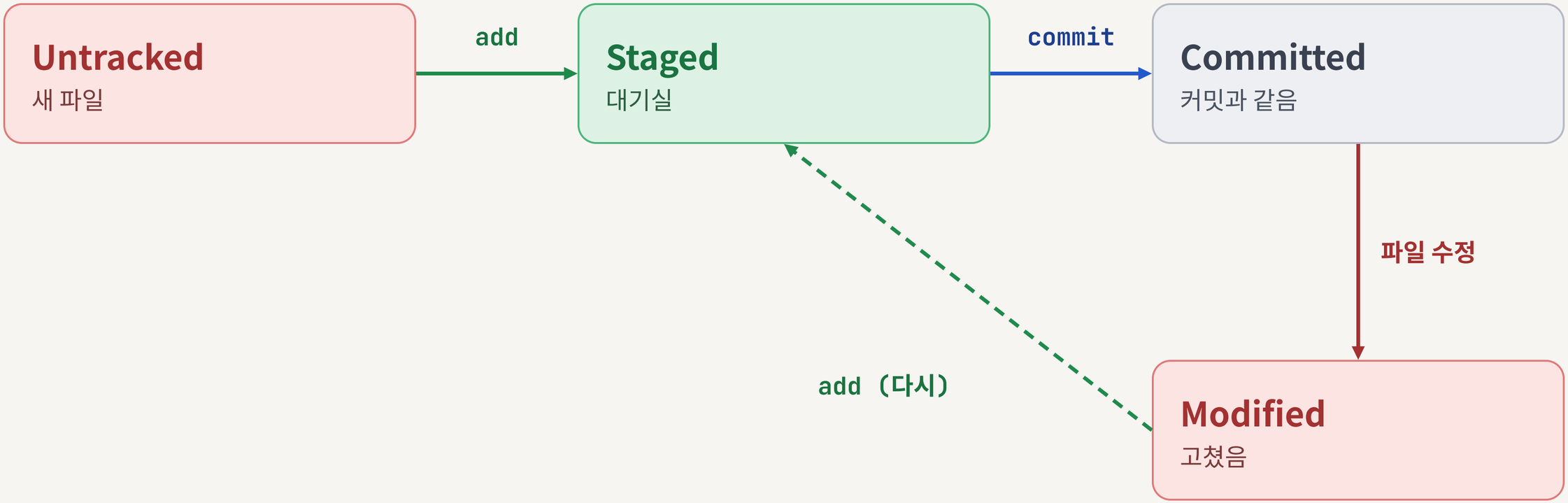
빨간 글씨 = 대기실 밖

초록 글씨 = 대기실 안

용어 ② 저장소와 꼬리표

용어	한 줄 뜻
Repository	커밋 기록 보관함 (.git, GitHub)
Index	대기실의 정식 이름
Commit · 해시	저장된 한 시점과 그 고유 번호
branch (브랜치)	커밋에 붙이는 이름표. 03에서 자세히 봐요
main	기본 브랜치. 최신 커밋에 붙은 꼬리표
HEAD	내가 지금 서 있는 곳
origin	GitHub 주소의 별명
origin/main	내 컴퓨터가 기억하는 GitHub main 위치

파일 하나의 한살이


Untracked
새 파일
add
Staged
대기실
commit
Committed
커밋과 같음
파일 수정
Modified
고쳤음
add (다시)

실습 0 · 컴퓨터마다 처음 한 번만

Git에게 내 이름표 알려주기

```
$ git config --global user.name "홍길동"
$ git config --global user.email "gildong@example.com"
$ git config --global init.defaultBranch main

$ git config --global --list
user.name=홍길동
user.email=gildong@example.com
init.defaultbranch=main
```

커밋마다 붙는
작성자 이름표

이메일은
GitHub 가입 메일과 같게

기본 브랜치 이름을
main으로

실습 1 · 혼자 기록하기

git init

내 컴퓨터

① 작업 폴더
hello-git/

빈 폴더

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

커밋 없음

새로 생김: .git 폴더

GitHub

④ 원격 저장소
(아직 없음)

연결 전

```
$ mkdir hello-git && cd hello-git
$ git init
Initialized empty Git repository in /Users/kim/hello-git/.git/
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

이 폴더를 Git이 지켜보기 시작

숨김 폴더 .git = ②③의 실체

실습 2 · 혼자 기록하기

echo "hello" > hello.txt

내 컴퓨터

① 작업 폴더

hello-git/

hello.txt

Untracked · 새 파일

② 대기실

.git/index

비어 있음

③ 내 저장소

hello-git/.git

커밋 없음

GitHub

④ 원격 저장소

(아직 없음)

연결 전

```
$ echo "hello" > hello.txt
$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt
nothing added to commit but untracked files present
```

Git이 처음 보는 파일

빨간 글씨 = 아직 기록 대상 아님

실습 3 · 혼자 기록하기

git add hello.txt

내 컴퓨터

① 작업 폴더
hello-git/
hello.txt
원본은 그대로

② 대기실
.git/index
hello.txt
Staged · 새 파일

③ 내 저장소
hello-git/.git
커밋 없음

GitHub

④ 원격 저장소
(아직 없음)
연결 전

```
$ git add hello.txt
# 아무 출력이 없으면 성공
$ git status
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file: hello.txt
```

지금 모습을 찍어 대기실에 올림

초록 글씨 = 커밋 준비 끝

실습 4 · 혼자 기록하기

git commit -m "feat: 인사 파일 추가"

내 컴퓨터

① 작업 폴더
hello-git/

hello.txt
Committed · 변경 없음

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

538dd06 main

GitHub

④ 원격 저장소
(아직 없음)

연결 전

```
$ git commit -m "feat: 인사 파일 추가"
[main (root-commit) 538dd06] feat: 인사 파일 추가
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
$ git status
On branch main
nothing to commit, working tree clean
```

538dd06 번으로 기록 완료

GitHub엔 아직 없음

실습 5 · 혼자 기록하기

echo "world" >> hello.txt

내 컴퓨터

① 작업 폴더

hello-git/

hello.txt

Modified · 수정됨

② 대기실

.git/index

비어 있음

③ 내 저장소

hello-git/.git

538dd06 main

GitHub

④ 원격 저장소

(아직 없음)

연결 전

```
$ echo "world" >> hello.txt
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
        modified: hello.txt
no changes added to commit (use "git add" and/or "git commit -a")
```

마지막 커밋과 달라진 걸 발견

다음: git add

실습 6 · 헷갈림 주의

git add hello.txt → echo "more" >> hello.txt




```
$ git add hello.txt
$ echo "more" >> hello.txt
$ git status
Changes to be committed:
        modified: hello.txt
Changes not staged for commit:
        modified: hello.txt
# 안내 문구(use ...)는 생략
```

지금 commit 하면 more는 빠져요

커밋 직전에 한 번 더 add

실습 7 · 혼자 기록하기

git add hello.txt → git commit -m "feat: 인사말 추가"

내 컴퓨터

① 작업 폴더
hello-git/

hello.txt
Committed · 변경 없음

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

a7c21f4 main

538dd06

GitHub

④ 원격 저장소
(아직 없음)

연결 전

```
$ git add hello.txt
$ git commit -m "feat: 인사말 추가"
[main a7c21f4] feat: 인사말 추가
 1 file changed, 2 insertions(+)
$ git log --oneline
a7c21f4 (HEAD → main) feat: 인사말 추가
538dd06 feat: 인사 파일 추가
```

커밋이 줄줄이 쌓여요

main은 늘 최신 커밋을 가리켜요

커밋 메시지 ① · 방금 쓴 feat: 는 뭘까?

한 달 뒤, 어느 쪽 기록을 읽고 싶나요?

아무렇게나

```
$ git log --oneline
9f1c2aa 수정
4b7e0d1 ㅇㅇ
c21a9f3 asdf
7d0e5b2 최종
1a2b3c4 진짜최종
```

규칙대로

```
$ git log --oneline
9f1c2aa fix: 로그인 버튼이 안 눌리는 문제 수정
4b7e0d1 feat: 회원가입 이메일 인증 추가
c21a9f3 docs: README에 설치 방법 추가
7d0e5b2 style: 들여쓰기 정리
1a2b3c4 refactor: 로그인 함수 분리
```

- 무엇이 바뀌었는지 바로 보여요
- 팀원이 리뷰하기 쉬워요
- 문제 생긴 시점을 찾기 쉬워요

커밋 메시지 ②

이렇게 써요: 타입 + 요약

feat : 회원가입 이메일 인증 추가

타입 · 콜론과 공백 · 무엇을 했는지

feat
새 기능

fix
버그 수정

docs
문서 (README 등)

style
모양만 정리

refactor
동작은 같게 구조 개선

test
테스트 코드

chore
설정, 패키지

한 커밋 = 한 가지 일
요약은 짧게

실습 8 · GitHub과 연결하기

GitHub에 빈 저장소 만들기

1

github.com
→ New

2

이름
hello-git

3

README 체크
해제

4

Create
→ 주소 복사

`https://github.com/kim/hello-git.git`

다음 장에서 씀

실습 9 · GitHub과 연결하기

git remote add origin https://github.com/kim/hello-git.git

내 컴퓨터

① 작업 폴더

hello-git/

hello.txt
Committed · 변경 없음

② 대기실

.git/index

비어 있음

③ 내 저장소

hello-git/.git

a7c21f4 main

538dd06

GitHub · 별명 origin

④ 원격 저장소

github.com/kim/hello-git

빈 저장소

주소에 별명 origin

```
$ git remote add origin https://github.com/kim/hello-git.git
# 출력 없음 = 등록 성공
$ git remote -v
origin https://github.com/kim/hello-git.git (fetch)
origin https://github.com/kim/hello-git.git (push)
```

주소록에 등록만 했어요

아직 보낸 건 없음

실습 10 · GitHub과 연결하기

git push origin main

내 컴퓨터

① 작업 폴더
hello-git/

hello.txt
Committed · 변경 없음

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

a7c21f4 main origin/main

538dd06

GitHub · 별명 origin

④ 원격 저장소
github.com/kim/hello-git

a7c21f4 main

538dd06

해시가 같아요 = 같은 커밋

```
$ git push origin main
Enumerating objects: 6, done.
Writing objects: 100% (6/6), 466 bytes | 466.00 KiB/s, done.
To https://github.com/kim/hello-git.git
 * [new branch] main → main
$ git log --oneline --graph --all
* a7c21f4 (HEAD → main, origin/main) feat: 인사말 추가
* 538dd06 feat: 인사 파일 추가
```

origin으로 main을 보냄

push 뒤에 어디로(origin) 무엇을(main)

실습 11 · 커밋만 하고 push 안 하면?

git commit -m "feat: 끝인사 추가"

내 컴퓨터

① 작업 폴더
hello-git/

hello.txt
Committed · 변경 없음

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

3f9a2c1 main
a7c21f4 origin/main
538dd06

1개 앞서 있음 (안 보냄)

GitHub · 별명 origin

④ 원격 저장소
github.com/kim/hello-git

a7c21f4 main
538dd06

```
$ git add hello.txt
$ git commit -m "feat: 끝인사 추가"
[main 3f9a2c1] feat: 끝인사 추가
$ git status
nothing to commit, working tree clean
$ git log --oneline --graph --all
* 3f9a2c1 (HEAD → main) feat: 끝인사 추가
* a7c21f4 (origin/main) feat: 인사말 추가
* 538dd06 feat: 인사 파일 추가
```

**status는 clean,
그래도 GitHub엔 없음**

main이 origin/main보다 위

git push origin main

내 컴퓨터

① 작업 폴더
hello-git/

hello.txt
Committed · 변경 없음

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

3f9a2c1 main origin/main
a7c21f4
538dd06

GitHub · 별명 origin

④ 원격 저장소
github.com/kim/hello-git

3f9a2c1 main
a7c21f4
538dd06

```
$ git push origin main
To https://github.com/kim/hello-git.git
   a7c21f4..3f9a2c1 main → main
$ git log --oneline --graph --all
* 3f9a2c1 (HEAD → main, origin/main) feat: 끝인사 추가
* a7c21f4 feat: 인사말 추가
* 538dd06 feat: 인사 파일 추가
```

main = origin/main
이제 같아요

실습 13 · 팀원 컴퓨터에서

git clone https://github.com/kim/hello-git.git




```
$ cd ~/projects
$ git clone https://github.com/kim/hello-git.git
Cloning into 'hello-git'...
Receiving objects: 100% (9/9), done.
$ cd hello-git
# 새로 생긴 폴더로 꼭 들어가기
$ ls
hello.txt
```

GitHub 저장소를 통째로 복사

init + remote add + 받기를 한 번에

실습 14 · 다시 내 컴퓨터 (팀원이 README를 push함)

git fetch origin

내 컴퓨터

① 작업 폴더
hello-git/

hello.txt
Committed · 변경 없음

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

e73617d origin/main
3f9a2c1 main
a7c21f4
538dd06

GitHub · 별명 origin

④ 원격 저장소
github.com/kim/hello-git

e73617d main
3f9a2c1
a7c21f4
538dd06

```
$ git fetch origin
From https://github.com/kim/hello-git
   3f9a2c1..e73617d main → origin/main
$ git log --oneline --graph --all
* e73617d (origin/main) docs: README 추가
* 3f9a2c1 (HEAD → main) feat: 끝인사 추가
* a7c21f4 feat: 인사말 추가
* 538dd06 feat: 인사 파일 추가
```

받아만 왔어요

origin/main만 이동
작업 폴더는 그대로

실습 15 · 받아서 합치기

git pull origin main




```
$ git pull origin main
From https://github.com/kim/hello-git
 * branch main → FETCH_HEAD
Updating 3f9a2c1..e73617d
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

받고 + 합치고

main이 따라오고
README.md가 생김

개념 정리

pull = fetch + merge



① `git fetch origin` 받아만 오기

② `git merge origin/main` 내 것과 합치기

`git pull origin main` = ① + ② 한 번에

init으로 시작? clone으로 시작?

GitHub에 아직 없음

```
git init
```

```
add → commit
```

```
git remote add origin 주소
```

```
git push origin main
```

GitHub에 이미 있음

```
git clone 주소
```

```
cd 폴더이름
```

```
add → commit
```

```
git push origin main
```

둘 중 하나만. 시작한 뒤에는 똑같아요.

되돌리기

커밋을 되돌릴 땐 push 했는지부터 봐요



③ 내 저장소에만 있음

push



④ GitHub에도 올라감

push 전 · 커밋 취소

```
git reset HEAD~1
```

잘못 지웠다면 git reflog로 다시 찾아요

push 후 · 되돌리는 커밋 추가

```
git revert 커밋
```

팀원 기록은 그대로 둬요

실습 16 · 되돌리기

git commit -m "feat: 실수한 커밋"

내 컴퓨터

① 작업 폴더

hello-git/

hello.txt
Committed · 변경 없음

README.md
Committed · 변경 없음

② 대기실

.git/index

비어 있음

③ 내 저장소

hello-git/.git

- b81d4e2 main
- e73617d origin/main
- 3f9a2c1
- a7c21f4
- 538dd06

GitHub · 별명 origin

④ 원격 저장소

github.com/kim/hello-git

- e73617d main
- 3f9a2c1
- a7c21f4
- 538dd06

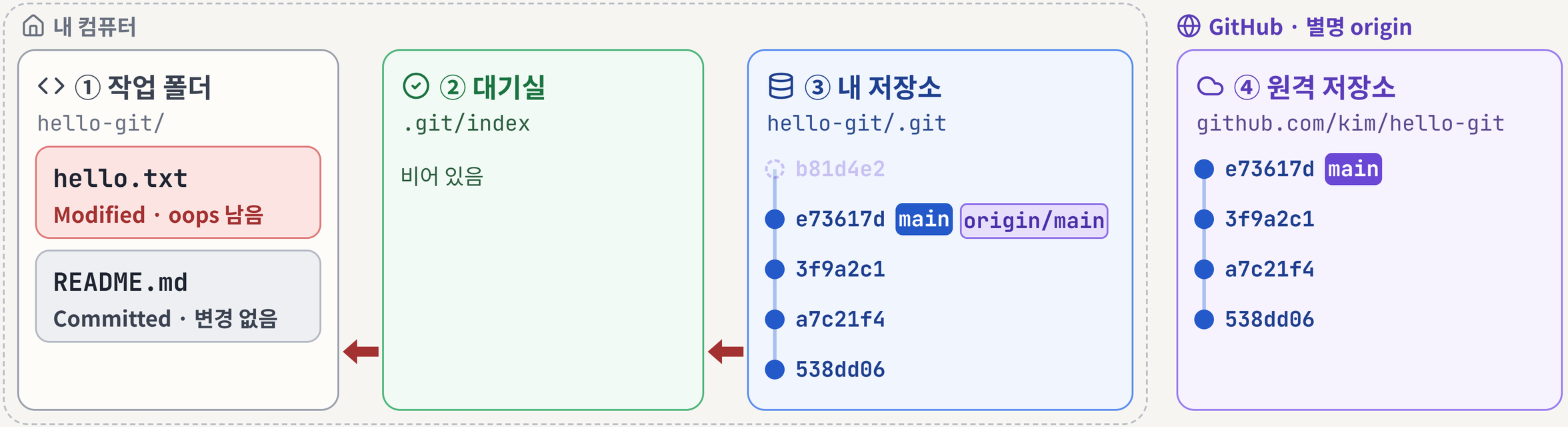
```
$ echo "oops" >> hello.txt
$ git add hello.txt
$ git commit -m "feat: 실수한 커밋"
[main b81d4e2] feat: 실수한 커밋
 1 file changed, 1 insertion(+)
$ git log --oneline --graph --all
* b81d4e2 (HEAD → main) feat: 실수한 커밋
* e73617d (origin/main) docs: README 추가
```

앗, 잘못 커밋했다

아직 push 전이에요

실습 17 · 되돌리기 (push 전)

git reset HEAD~1




```
$ git reset HEAD~1
Unstaged changes after reset:
M       hello.txt
$ git log --oneline --graph --all
* e73617d (HEAD → main, origin/main) docs: README 추가
* 3f9a2c1 feat: 끝인사 추가
# HEAD~1 = 지금보다 한 칸 아래 커밋
```

main을 한 칸 뒤로

고친 내용은 작업 폴더에 남아요

되돌리기 · reset 옵션 3가지

커밋한 변경은 어디로 갈까?

	③ 내 저장소	② 대기실	① 작업 폴더
`--soft`	커밋 취소	변경이 여기 남음	그대로
`--mixed` (기본)	커밋 취소	비움	변경이 여기 남음
`--hard`	커밋 취소	비움	변경이 사라짐

되돌리기 · 지운 커밋 다시 찾기

reset으로 지운 커밋, git reflog로 찾아요




```
$ git reflog
e73617d HEAD@{0}: reset: moving to HEAD~1
b81d4e2 HEAD@{1}: commit: feat: 실수한 커밋
e73617d HEAD@{2}: pull origin main: Fast-forward
3f9a2c1 HEAD@{3}: commit: feat: 끝인사 추가
```

```
$ git reset --hard b81d4e2
HEAD is now at b81d4e2 feat: 실수한 커밋
```

HEAD가 움직인 기록이 순서대로 남아요

내 컴퓨터에만 있고, GitHub엔 없어요

실습 18 · 만약 push까지 했다면?

git revert HEAD → git push origin main

내 컴퓨터

① 작업 폴더
hello-git/

hello.txt
Committed · 변경 없음

README.md
Committed · 변경 없음

② 대기실
.git/index

비어 있음

③ 내 저장소
hello-git/.git

- c4e9a07 main origin/main
- b81d4e2
- e73617d
- 3f9a2c1
- a7c21f4

GitHub · 별명 origin

④ 원격 저장소
github.com/kim/hello-git

- c4e9a07 main
- b81d4e2
- e73617d
- 3f9a2c1
- a7c21f4

```
# b81d4e2(실수한 커밋)을 이미 push한 상태
$ git revert HEAD
[main c4e9a07] Revert "feat: 실수한 커밋"
 1 file changed, 1 deletion(-)
$ git push origin main
To https://github.com/kim/hello-git.git
   b81d4e2..c4e9a07 main → main
```

지우지 않고
되돌리는 커밋 추가

팀원 기록을 망치지 않아요

되돌리기 · 정리

reset은 지우고, revert는 덧붙여요

reset



main을 B로 되감아요
X는 기록에서 빠짐

push 전, 나만 가진 커밋

revert



X를 지우지 않고
반대로 고친 커밋을 추가

push 후, 팀과 공유된 커밋

막히면 git status부터

이 문장이 보이면	지금 상태	다음 할 일
No commits yet	커밋 0개	add → commit
Untracked files:	① 새 파일	git add
Changes not staged for commit:	① 고쳤지만 add 전	git add
Changes to be committed:	② 커밋 대기	git commit
nothing to commit, working tree clean	모두 커밋됨	push 했는지 확인

GitHub과 비교는 log로

`git fetch origin → git log --oneline --graph --all`

main이 위 → push origin main

origin/main이 위 → pull origin main

에러 1 · index.lock

index.lock = Git의 ‘사용 중’ 팻말

명령 시작

팻말 걸기

→

작업 중

다른 명령은 대기

→

명령 끝

팻말 떼기

끝나기 전에 멈추면 팻말이 남아요

창 강제 종료 · 안 닫은 커밋 에디터 · VS Code가 동시에 git 실행

에러 1 · index.lock

이 메시지가 보이면

```
$ git add hello.txt
fatal: Unable to create '/Users/kim/hello-git/.git/index.lock': File exists.
Another git process seems to be running in this repository, ...
```

1

다른 git 작업
끝났는지 확인

2

팻말 떼기

```
Mac: rm .git/index.lock
```

```
Windows: del .git\index.lock
```

3

아까 명령
다시 실행

지우는 건 index.lock 하나만. index나 .git 폴더는 절대 지우지 않아요.

에러 2 · add 없이 commit

commit 했는데 아무 일도 없어요

```
$ git commit -m "feat: 새 파일"
Untracked files:
        new.txt
nothing added to commit but untracked
files present

$ git commit -m "fix: 오타 수정"
Changes not staged for commit:
        modified:   hello.txt
no changes added to commit
```



② 대기실이 비어 있어요

```
git add 파일
```

```
git commit -m "..."
```

에러 3 · origin 설정

origin이 이미 있대요, 혹은 없대요

두 번 등록함

```
error: remote origin already exists.
```

```
git remote -v
```

```
git remote set-url origin 새주소
```

등록 안 함

```
fatal: 'origin' does not appear to be a git repository
```

```
git remote add origin 주소
```

```
git push origin main
```

에러 4 · 어디로 보낼지 안 적음

git push만 쳤더니 거절돼요

```
$ git push
fatal: The current branch main has no upstream branch.
$ git pull
There is no tracking information for the current branch.
```

우리 과정 규칙: 어디로, 무엇을 항상 적기

```
git push origin main
git pull origin main
```

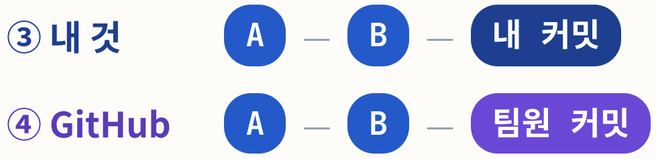
⚠

--set-upstream(-u) 안내가 떠도 쓰지 않아요

에러 5 · push 거절

push 했더니 rejected

```
$ git push origin main
 ! [rejected]        main → main (fetch first)
error: failed to push some refs to 'https://github.com/kim/hello-git.git'
```



서로 모르는 커밋이 생겼어요

```
1. git pull origin main
```

```
2. git push origin main
```

--force는 쓰지 않아요

한 장 요약

① 작업 폴더 → ② 대기실 → ③ 내 저장소 → ④ GitHub

add → commit → push origin main

pull origin main

새로 시작

```
git init
add → commit
git remote add
  origin 주소
git push origin main
```

GitHub에서 받기

```
git clone 주소
cd 폴더이름
```

매일 반복

```
git pull origin main
(작업)
git status
git add .
git commit -m
  "feat: ..."
git push origin main
```

되돌리기

```
git reset HEAD~1
git revert 커밋
git reflog
```